United International University

Department of Computer Science and Engineering

CSE 1115: Object Oriented Programming — Final: Spring 2026

Total Marks: 40 — Time: 2 hours

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

Answer all five questions. The numbers on the right of the questions denote their marks. Wish you best of luck.

1. In the following code only one class is missing. Complete **that class only** so that the specified output is obtained. (8)

Main.java

```
interface FastMovable {
    void move();
}

interface SlowMovable {
    String move();
}

abstract class Character {
    String name;
    Character(String name) {
        this.name = name;
    }
    abstract void move();
}

// a class is missing, write it correctly

public class Main {
    public static void main(String[] args) {
        Player p1 = new Player("Bokul");
        Character p2 = new Player("Chameli");
        FastMovable p3 = new Player("Dahlia");
        p1.move();
        p2.move();
        p3.move();
    }
}
```

Output:

```
Bokul moves
Chameli moves
Dahlia moves
```

2. Observe the following program and answer parts (a) and (b). (2+6 = 8)

AudioProcessor.java

```
public class AudioProcessor {
    public static void checkVolume(String dbStr) throws AudioOverloadException {
        try {
            int db = Integer.parseInt(dbStr);
            System.out.println("Check: Try-Start");
            if (db > 120) throw new AudioOverloadException("ClipRisk");
            System.out.println("Check: Try-End");
        } catch (NumberFormatException e) {
            System.out.println("Check: Catch NFE");
        } finally {
            System.out.println("Check: Finally");
        }
        System.out.println("Check: Outside");
    }

    public static void main(String[] args) {
        try {
            System.out.println("Main: Try1");
            checkVolume("mute");
            System.out.println("Main: Try2");
        } catch (AudioOverloadException e) {
            System.out.println("Main: Catch AOE1 " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Main: Catch NFE");
            try {
                System.out.println("Main: Nested-Try1");
                checkVolume("130");
                System.out.println("Main: Nested-Try2");
            } catch (AudioOverloadException ex) {
                System.out.println("Main: Nested-Catch AOE " + ex.getMessage());
            }
        } finally {
            System.out.println("Main: Finally");
        }
        System.out.println("Main: Outside");
    }
}
```

(a) Write the complete class definition for the custom exception `AudioOverloadException` exactly as it is utilized in the code snippet above.

(b) Trace the control flow carefully and determine the **exact line-by-line console output** produced when the program executes.

3. You are implementing the core transaction logic for a bank account management interface. The GUI provides a balance display label (**Balance**), a numeric input text field (**TextField**), and two buttons: **Deposit** and **Withdraw**. (4+2+2 = 8)

- **Balance** initially displays the text "`5000.00`" and must always be shown with **exactly two decimal places** (currency is a decimal value, not an integer).
- **TextField** accepts a string representing the transaction amount.

When a button is pressed, the application must read the amount from `TextField`, apply the operation to the current value in `Balance`, and update `Balance` with the new total. You must write **only one** `actionPerformed(ActionEvent e)` method, using the `ActionEvent` parameter to determine which button fired. Assume all component references (`Balance`, `TextField`, `Deposit`, `Withdraw`) are accessible as class-level fields. Your method must satisfy the following rules:

(a) Identify the source button from the `ActionEvent` and perform the matching operation (deposit *adds*, withdraw *subtracts*). Update `Balance`, formatted to two decimal places.

(b) **Input validation.** If `TextField` does not contain a valid number, the operation must *not* change the balance; instead set `Balance` to display `Invalid Amount`. Handle this case using exception handling (do not let the program crash).

(c) **Overdraft guard.** A withdrawal may never drive the balance below zero. If the requested amount exceeds the current balance, reject the transaction, leave the balance unchanged, and display `Insufficient Funds`. (Deposits are never rejected.)

Sample behaviour (starting from a balance of `5000.00`):

```
Deposit "250.50" -> Balance:  5250.50
Withdraw "750" -> Balance:  4500.50
Withdraw "999999" -> Balance:  Insufficient Funds
Deposit "abc" -> Balance:  Invalid Amount
```

4. The Galactic Republic needs to organize its fleet. You are given a class `Starship`. (4+4+2 = 10)

EligibilityList.java

```java
class Starship implements Comparable<Starship> {
    String name;
    int powerRating;
    String status;

    public Starship(String name, int powerRating, String status) {
        this.name = name;
        this.powerRating = powerRating;
        this.status = status;
    }

    public String toString() {
        return name + " " + powerRating + " " + status;
    }

    // Implement the compareTo method
}

public class Main {
    public static void main(String[] args) {
        // Write code here
```

```
    }
}
```

Write the missing code in `Starship` and `Main` to perform the following tasks:

(a) Create an `ArrayList<Starship>` called `fleet` inside `main`. Read the ship data from a file named `starships.txt`, create corresponding objects, and insert those into the `ArrayList`. Note that, `starships.txt` contains the following lines (One Starship per line, space separated.)

```
X-Wing 85 Active
TIE_Fighter 30 Decommissioned
Millennium_Falcon 95 Active
Star_Destroyer 95 Active
Y-Wing 65 Reserve
Imperial_Shuttle 50 Decommissioned
```

(b) Sort the list `fleet` in descending order based on `powerRating`. If two starships have the same power rating, sort them alphabetically by their `name`. Save the sorted `ArrayList` into a new output file named `sorted_fleet.txt`.

(c) Remove all starships with a `powerRating` less than 60. Save the details of the remaining ships of the `fleet` into a new output file named `new_fleet.txt`.

5. Suppose you have two integer arrays representing the marks obtained by students from two different sections. You need to create two threads to calculate the sums in parallel. Ensure both threads finish execution before printing the results and handle any `InterruptedException` that might occur. **Note:** You only need to write the code corresponding to the commented sections. (6)

StudentMarks.java

```java
/** Write a class named "Calculator" **/

public class StudentMarks {
    public static void main(String[] args) {
        int[] sectionA = {80, 75, 90, 85};
        int[] sectionB = {70, 88, 92, 76};

        Calculator threadA = new Calculator(sectionA);
        Calculator threadB = new Calculator(sectionB);

        /** Write code for parallely executing functions to calculate sum **/

        System.out.println("Section A Total = " + threadA.getTotal());
        System.out.println("Section B Total = " + threadB.getTotal());
    }
}
```

Sample Output:

```
Section A Total = 330
Section B Total = 326
```